



Caso 1

Construcción y Mantenimiento de Software

GRUPO 2

MILTON ALVARADO RAMIREZ

MICHAEL JIMENEZ MONTERO

TOMÁS AZOFEIFA QUESADA

RODOLFO CAMPOS SOLANO

BRANDON GERARDO GARITA ARROYO

Universidad CENFOTEC

CURSO PSWE-03

Prof. Francisco Vargas Navarro

06 junio 2026

Tabla de contenido

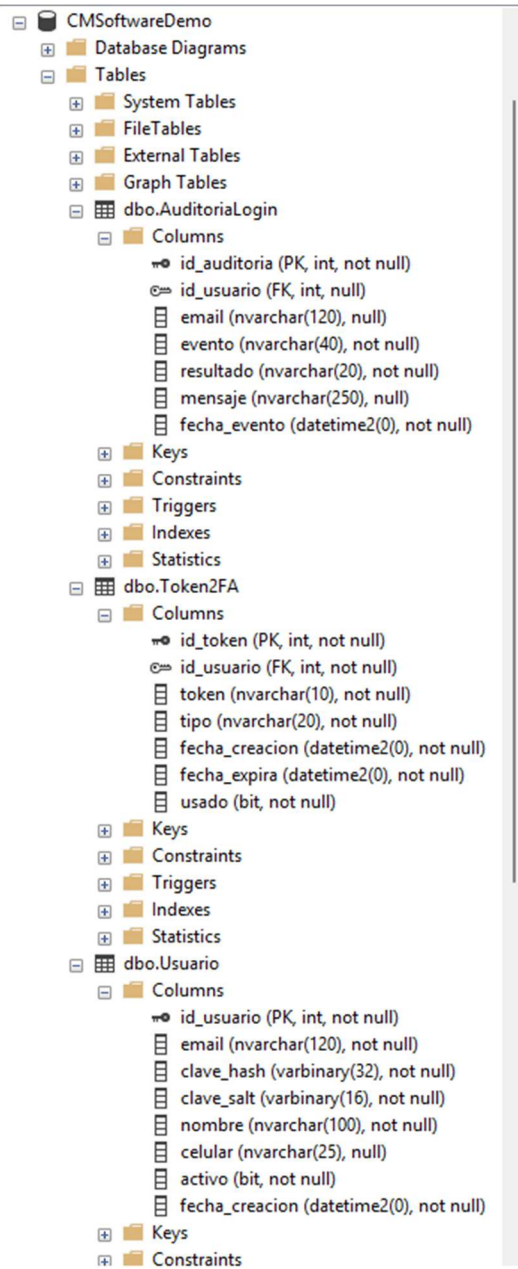
1. Introducción	3
2. Evidencias de ejecución.....	4
2.1 Base de datos creada.....	4
2.2 Pantalla de login.....	6
2.3 Token enviado o mostrado en consola	7
2.4 Menú principal	9
2.4 Registro de usuario	10
2.5 Auditoría de accesos	11
3. Tabla de pruebas ejecutadas	11
4. Defectos encontrados	14
5. Mejoras implementadas	15
Mejora de la interfaz gráfica	15
Beneficio principal.....	16
Auditoría personal de accesos	16
Beneficio principal.....	16
Ampliación y validación de pruebas unitarias	17
Beneficio principal.....	17
6. Reflexión sobre construcción y mantenimiento	17
7. Preguntas de reflexión	19
¿Qué parte del programa sería más difícil de mantener si el sistema creciera?	19
¿Qué riesgos existen al guardar la clave en texto plano?.....	20
¿Por qué conviene separar la interfaz gráfica del acceso a datos?	21
¿Qué parte debería automatizarse mediante pruebas unitarias?.....	21
¿Qué cambiaría si este sistema fuera para producción real?	22

1.Introducción

La práctica consistió en revisar, ejecutar, mejorar y validar un sistema de autenticación desarrollado en Python con Tkinter y SQL Server. El sistema incluye login, validaciones de entrada, clave protegida mediante hash + salt, recuperación de clave, doble autenticación por email, auditoría de accesos y pruebas unitarias.

2. Evidencias de ejecución

2.1 Base de datos creada



SQLQuery1.sql - S...wareDemo (sa (68))

```

select * from Usuario
select * from Token2FA
select * from AuditoriaLogin

```

100 %

Results Messages

	id_usuario	email	clave_hash	clave_salt	nombre	celular	activo	fecha_creacion
1	1	demo@fvnr.org	0x904EA76B7ACD70D9FFF7F23CAD89454EB1DCD4950F768F...	0x957814C951972C2E0FF833EB0FDBDC09	Usuario Demo	8888-8888	1	2026-06-05 18:42:49

	id_token	id_usuario	token	tipo	fecha_creacion	fecha_expira	usado
1	1	1	111797	LOGIN_2FA	2026-06-05 18:46:14	2026-06-05 18:51:14	1
2	2	1	567611	RECUPERACION	2026-06-05 18:46:40	2026-06-05 18:51:40	1
3	3	1	893783	RECUPERACION	2026-06-05 18:47:12	2026-06-05 18:52:12	1
4	4	1	548773	LOGIN_2FA	2026-06-05 18:47:24	2026-06-05 18:52:24	1
5	5	1	177133	LOGIN_2FA	2026-06-05 19:23:58	2026-06-05 19:28:58	1
6	6	1	414847	LOGIN_2FA	2026-06-05 19:25:00	2026-06-05 19:30:00	1

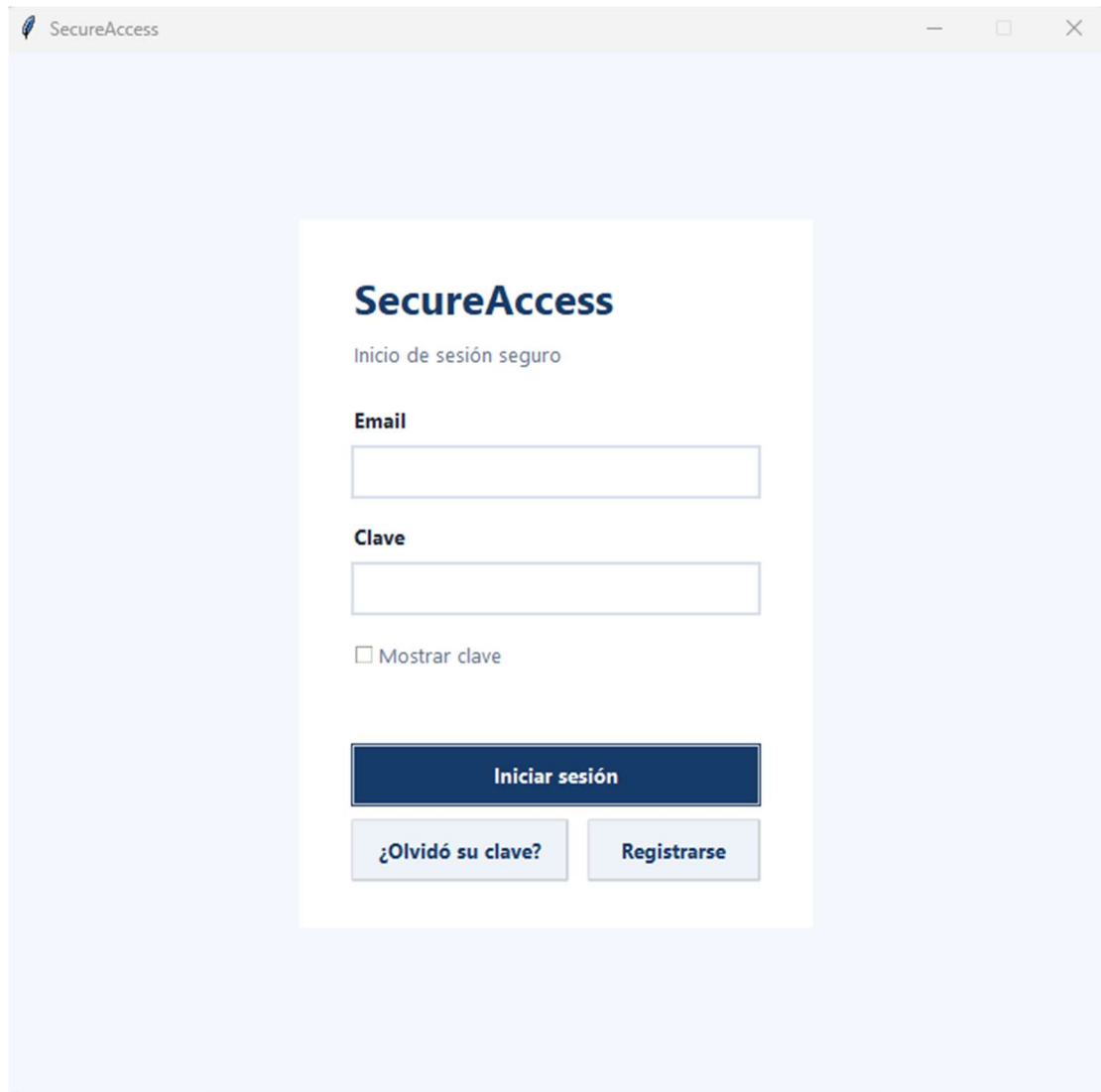
	id_auditoria	id_usuario	email	evento	resultado	mensaje	fecha_evento
1	1	1	demo@fvnr.org	LOGIN	EXITOSO	Credenciales válidas.	2026-06-05 18:46:14
2	2	1	demo@fvnr.org	LOGIN_2FA	EXITOSO	Token 2FA válido.	2026-06-05 18:46:20
3	3	1	demo@fvnr.org	MENU	EXITOSO	Opción consultada: Inicio	2026-06-05 18:46:22
4	4	1	demo@fvnr.org	MENU	EXITOSO	Opción consultada: Opción 1	2026-06-05 18:46:24
5	5	1	demo@fvnr.org	MENU	EXITOSO	Opción consultada: Opción 3	2026-06-05 18:46:25
6	6	1	demo@fvnr.org	MENU	EXITOSO	Opción consultada: Opción 2	2026-06-05 18:46:26
7	7	1	demo@fvnr.org	RECUPER...	EXITOSO	Token de recuperación gen...	2026-06-05 18:46:40
8	8	1	NULL	CAMBIO_...	EXITOSO	Clave actualizada correcta...	2026-06-05 18:47:01
9	9	1	demo@fvnr.org	RECUPER...	EXITOSO	Token de recuperación gen...	2026-06-05 18:47:12
10	10	1	NULL	CAMBIO_...	EXITOSO	Clave actualizada correcta...	2026-06-05 18:47:19
11	11	1	demo@fvnr.org	LOGIN	EXITOSO	Credenciales válidas.	2026-06-05 18:47:24
12	12	1	demo@fvnr.org	LOGIN_2FA	EXITOSO	Token 2FA válido.	2026-06-05 18:47:29
13	13	1	demo@fvnr.org	LOGIN	EXITOSO	Credenciales válidas.	2026-06-05 19:23:58
14	14	1	demo@fvnr.org	LOGIN_2FA	EXITOSO	Token 2FA válido.	2026-06-05 19:24:44
15	15	1	demo@fvnr.org	LOGIN	EXITOSO	Credenciales válidas.	2026-06-05 19:25:00
16	16	1	demo@fvnr.org	LOGIN_2FA	EXITOSO	Token 2FA válido.	2026-06-05 19:25:36

Captura de la base de datos CMSSoftwareDemo y de sus tablas principales.

Nota: la captura muestra la estructura de tablas, no los valores de credenciales protegidas ni los tokens.

La base de datos permite almacenar información de usuarios, credenciales protegidas, tokens de autenticación, claves de recuperación y registros de auditoría de inicio de sesión.

2.2 Pantalla de login



SecureAccess

SecureAccess

Inicio de sesión seguro

Email

Clave

☐ Mostrar clave

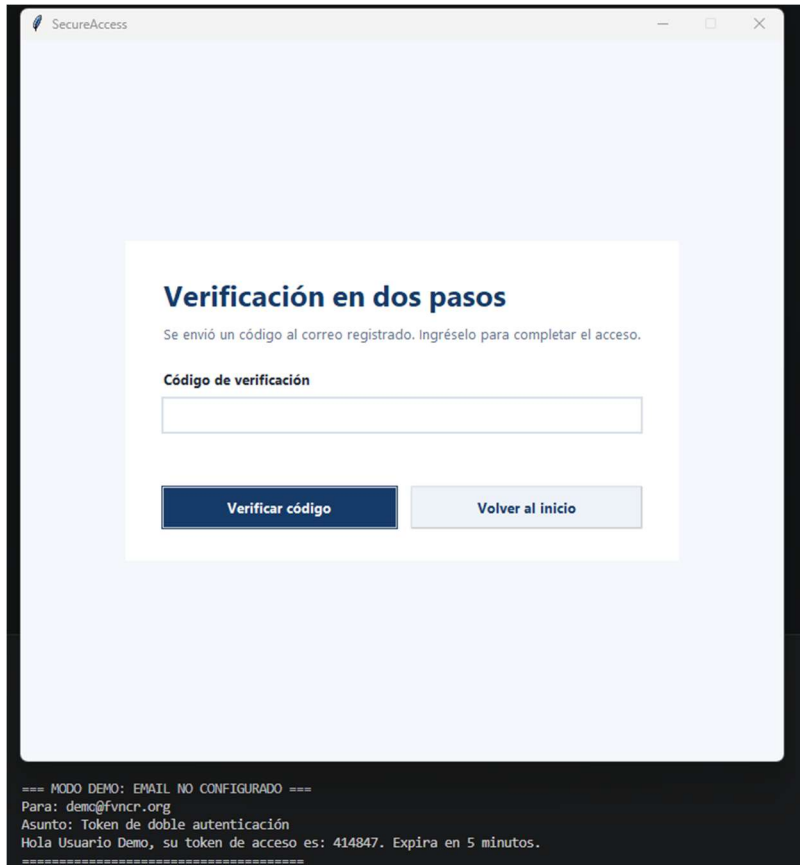
Iniciar sesión

[¿Olvidó su clave?](#) [Registrarse](#)

Captura de pantalla del inicio de sesión de SecureAccess.

La pantalla permite ingresar el correo electrónico y la contraseña. Valida los campos obligatorios, el formato de correo electrónico y los caracteres permitidos antes de iniciar la autenticación.

2.3 Token enviado o mostrado en consola



SecureAccess

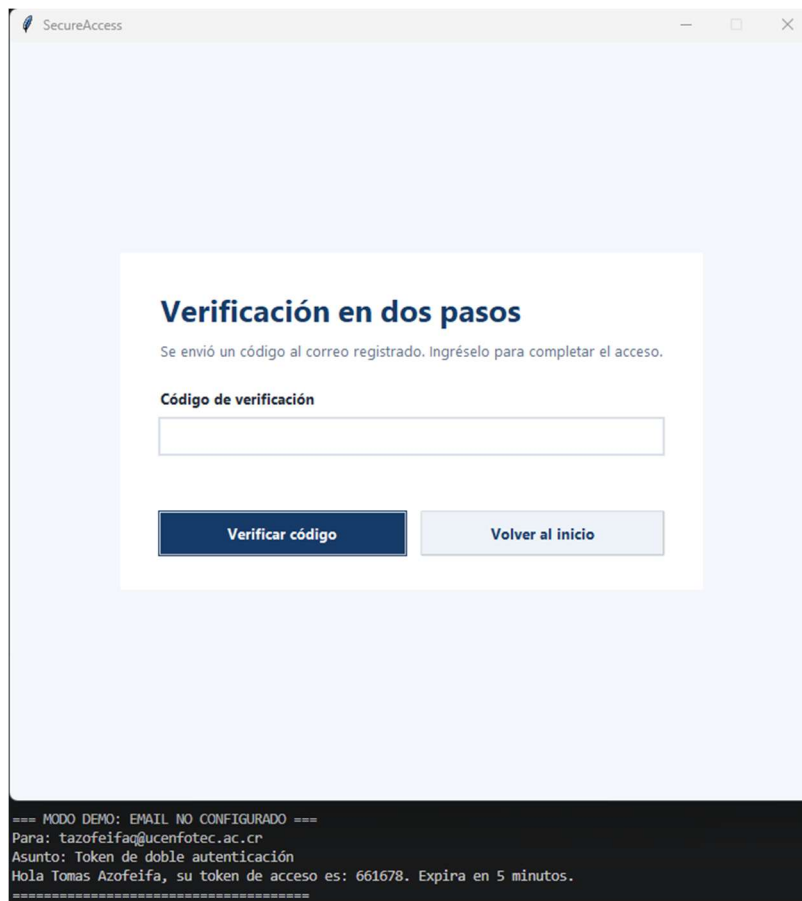
Verificación en dos pasos

Se envió un código al correo registrado. Ingrésele para completar el acceso.

Código de verificación

Verificar código Volver al inicio

=== MODO DEMO: EMAIL NO CONFIGURADO ===
Para: demo@fvncr.org
Asunto: Token de doble autenticación
Hola Usuario Demo, su token de acceso es: 414847. Expira en 5 minutos.

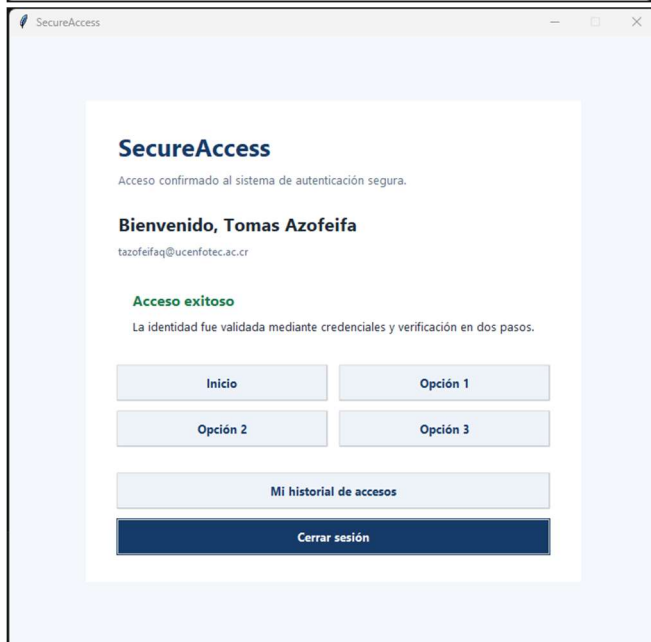
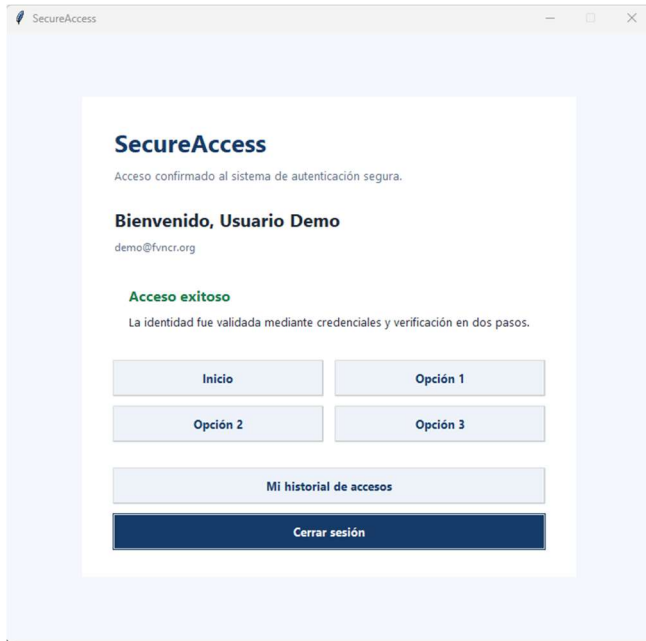


Captura de pantalla de la verificación 2FA y de la consola en modo demo.

Nota de seguridad: la captura contiene un token académico de ejecución local. Si el documento se comparte fuera del curso, se recomienda ocultarlo o difuminarlo.

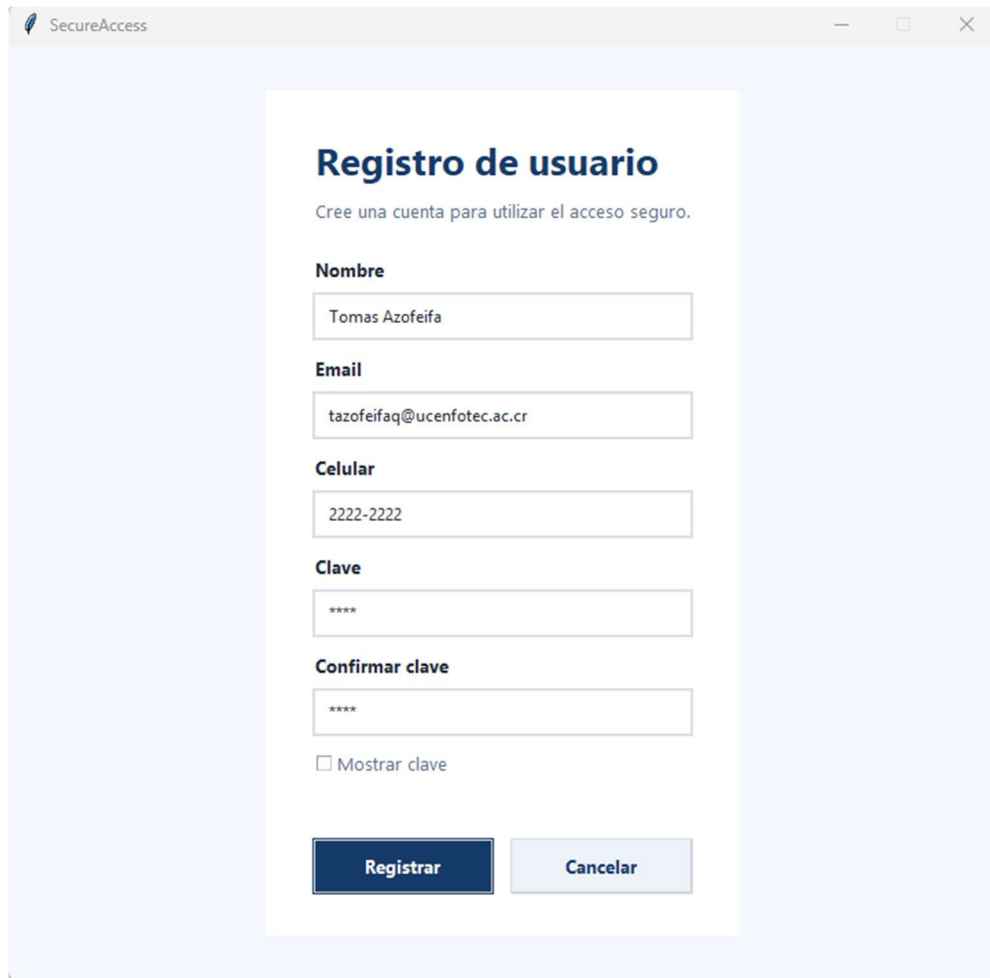
Tras un inicio de sesión correcto, el sistema genera un token de doble autenticación. Este token se envía por correo electrónico o se muestra en la consola en modo académico, según la configuración.

2.4 Menú principal



Captura del menú principal tras el inicio de sesión y la verificación correcta de 2FA. El menú principal se muestra únicamente después de completar correctamente el inicio de sesión y la validación de 2FA.

2.4 Registro de usuario



A screenshot of a web browser window titled "SecureAccess". The main content is a registration form titled "Registro de usuario" with the instruction "Cree una cuenta para utilizar el acceso seguro." The form contains several input fields: "Nombre" with the value "Tomas Azofeifa", "Email" with "tazofeifa@ucenfotec.ac.cr", "Celular" with "2222-2222", "Clave" with "****", and "Confirmar clave" with "****". There is a checkbox labeled "Mostrar clave" which is unchecked. At the bottom are two buttons: "Registrar" (dark blue) and "Cancelar" (light blue).

SecureAccess

Registro de usuario

Cree una cuenta para utilizar el acceso seguro.

Nombre

Email

Celular

Clave

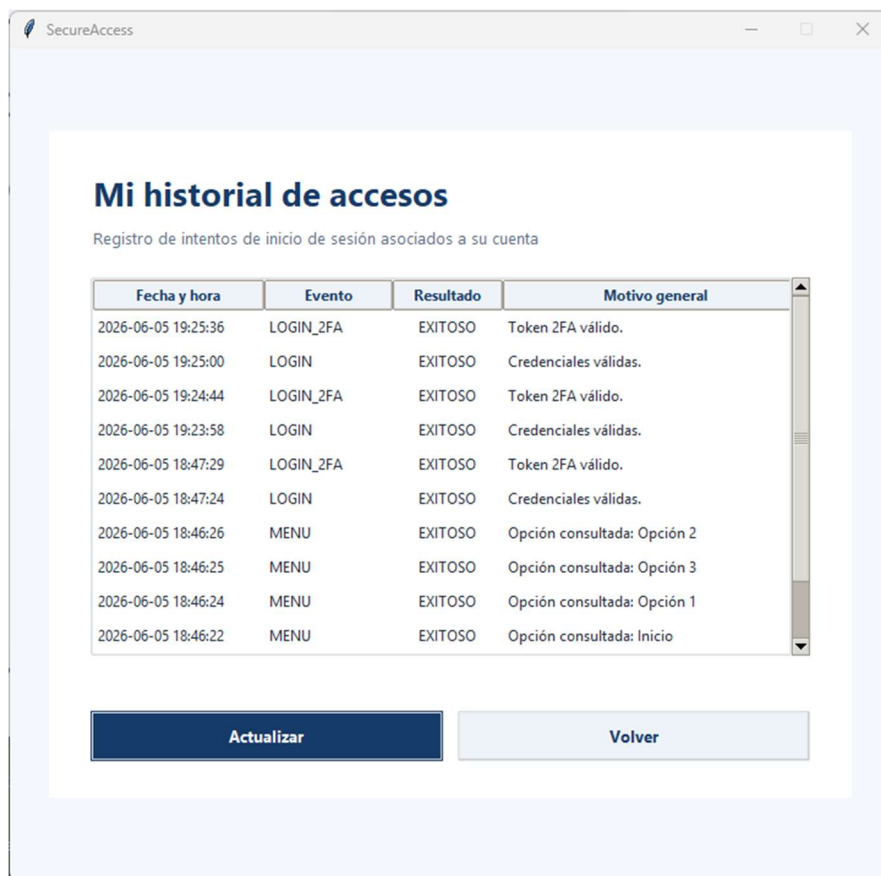
Confirmar clave

☐ Mostrar clave

Registrar **Cancelar**

Captura del registro de usuario

2.5 Auditoría de accesos



Captura de la auditoría de accesos del personal.

La auditoría permite consultar los registros asociados al usuario autenticado, lo que fortalece la trazabilidad y el control de accesos.

3. Tabla de pruebas ejecutadas

Comando ejecutado: `py -m pytest`. Resultado: 59 pruebas ejecutadas, 59 exitosas, 0 fallidas, 0 pendientes.

Nombre de la prueba	Descripción	Resultado final
test_email_invalido_por_formato[]	Verifica que un email vacío sea rechazado por la validación.	Exitosa

test_login_form_clave_vacia_produce_error	Verifica que una clave vacía sea rechazada antes del inicio de sesión.	Exitosa
test_email_invalido_por_formato[abc]	Verifica que un email con formato inválido sea rechazado.	Exitosa
test_login_form_clave_con_caracter_no_permitido_produce_error	Verifica que una clave con caracteres no permitidos sea rechazada.	Exitosa
test_usuario_inexistente_produce_error_controlado	¿Verifica que un usuario inexistente no pueda autenticarse y que se registre una auditoría?	Exitosa
test_clave_incorrecta_produce_error_controlado	Verifica que, al ingresar una contraseña incorrecta, se rechace el inicio de sesión y se registre una auditoría.	Exitosa
test_login_correcto_retorna_usuario_y_audita_exito	Verifica que las credenciales válidas devuelvan	Exitosa

	el usuario y que auditen con éxito.	
test_login_correcto_solicita_token_2fa	Verifica que un inicio de sesión correcto solicite o genere un token de 2FA.	Exitosa
test_verify_login_token_incorrecto_retorna_false_y_audita	Verifica que un token de 2FA incorrecto sea rechazado.	Exitosa
test_verify_login_token_correcto_retorna_true_y_audita	Verifica que un token de 2FA válido sea aceptado y auditado.	Exitosa
test_recuperacion_email_valido_genera_token_y_envia_correo	¿Verifica la recuperación con un email válido, un token y un correo electrónico simulado?	Exitosa
test_token_correcto_actualiza_clave_con_hash_y_salt	Verifica que el cambio de clave válido actualice la credencial protegida.	Exitosa
test_auditoria_personal_filtra_por_usuario_autenticado	Verifica que la auditoría personal se filtre según el usuario autenticado.	Exitosa

test_auditoria_registra_intento_exitoso_sin_datos_sensibles	¿Verifica la auditoría de inicio de sesión exitoso sin valores sensibles?	Exitosa
test_auditoria_registra_intento_fallido_sin_datos_sensibles	¿Verifica la auditoría de login fallido sin valores sensibles?	Exitosa
test_hash_no_es_igual_a_password_en_texto_plano	Verifica que la contrase?a no se represente como texto plano.	Exitosa
test_misma_password_con_salts_distintos_genera_hashes_distintos	Verifica que distintos saltos produzcan resultados distintos.	Exitosa

4. Defectos encontrados

ID	Defecto encontrado	Impacto	Corrección aplicada
D-01	Interfaz gráfica inicial demasiado simple o poco clara.	Dificultaba la experiencia del usuario y la presentación académica.	¿Se rediseña la interfaz con pantallas más claras, un estilo coherente y mensajes más comprensibles?
D-02	¿No existía una forma visual para consultar la auditoría personal?	El usuario no podía consultar su historial en la aplicación.	¿Se agregó la opción "Mi historial de accesos" al menú? principal.
D-03	Riesgo de exposición de	¿Una auditoría general podría	¿La consulta se filtró por el

	auditoría por parte de otros usuarios.	mostrar eventos de otras cuentas?	id_usuario del usuario autenticado?
D-04	Cobertura insuficiente de pruebas unitarias.	Las reglas críticas podían depender de la validación manual.	Se ampliaron las pruebas para validaciones, login, 2FA, recuperación, credenciales protegidas y auditoría.
D-05	Mensajes de error y textos visuales mejorables.	¿Podrán confundir al usuario o afectar la presentación?	Se normalizaron los mensajes y los textos con ortografía correcta y con errores controlados.

5. Mejoras implementadas

Mejora de la interfaz gráfica

Una de las principales mejoras fue el rediseño de la interfaz gráfica para hacerla más clara, moderna y consistente. La versión inicial de una aplicación funcional puede cumplir con los requisitos técnicos, pero aun así puede resultar difícil de presentar, usar o entender para un usuario final. En un sistema de autenticación, la claridad visual es especialmente importante porque el usuario debe comprender en qué etapa se encuentra: ingreso de credenciales, validación de 2FA, recuperación de clave, cambio de contraseña o acceso exitoso al menú principal.

La mejora de la interfaz permitió organizar mejor las pantallas y darles una apariencia más profesional. Se trabajó en elementos como el login, la recuperación de clave, la pantalla de verificación en dos pasos, los mensajes de error y el menú principal. Esto ayuda a que la aplicación no se perciba únicamente como un prototipo técnico, sino como un sistema más completo y presentable para una práctica académica.

En la pantalla de login se mejoró la disposición de los campos, los botones y los mensajes. También se agregó una opción para mostrar u ocultar la clave, lo que mejora la usabilidad sin comprometer la seguridad visual por defecto. La validación con la tecla Enter en el login y en la pantalla de 2FA también mejora la experiencia, porque permite completar el flujo de autenticación de forma más natural.

En las pantallas de recuperación y de cambio de contraseña, la interfaz se volvió más guiada. Esto es importante porque el usuario debe entender que primero solicita un código, luego lo

ingresa y, finalmente, establece una nueva clave. Además, los mensajes de error se hicieron más comprensibles y se evitó mostrar al usuario final información técnica innecesaria.

El principal beneficio de esta mejora es que facilita el uso del sistema y mejora la presentación académica. Una interfaz más ordenada permite obtener evidencias más claras, reduce la confusión durante la demostración y facilita explicar cada paso del proceso de autenticación.

Beneficio principal

Facilita el uso, mejora la presentación y aclara el flujo de autenticación.

Auditoría personal de accesos

Otra mejora importante fue la incorporación de una opción para consultar el historial de accesos del usuario autenticado. Antes de esta mejora, el sistema registraba eventos de auditoría en la base de datos, pero el usuario no contaba necesariamente con una interfaz visual para consultar sus propios registros en la aplicación. Esto limitaba el valor práctico de la auditoría para el usuario final.

La mejora consistió en agregar una opción en el menú principal llamada `Mi historial de accesos`. Esta pantalla permite mostrar los eventos asociados al usuario que inició sesión correctamente y completó la validación de 2FA. La auditoría se presenta en una tabla visual con información segura, como la fecha, el evento, el resultado y el motivo general.

El aspecto más relevante de esta mejora es el filtrado por el usuario autenticado. La consulta no depende de un email escrito manualmente ni de un selector visible para el usuario. En su lugar, utiliza el identificador del usuario ya autenticado mediante el flujo de login y 2FA. Esto evita que una persona consulte los registros de otras cuentas y reduce el riesgo de exposición de información.

Además, se amplió la auditoría para registrar los eventos del menú principal. Cuando el usuario utiliza opciones como `Inicio`, `Opción 1`, `Opción 2` u `Opción 3`, se registra la acción correspondiente. Esto mejora la trazabilidad porque no solo se registran los intentos de acceso, sino también ciertas interacciones posteriores dentro del sistema.

El principal beneficio es que la auditoría deja de ser únicamente un registro interno de la base de datos y se convierte en una funcionalidad visible y útil. Esto fortalece la transparencia para el usuario, facilita la evidencia académica y demuestra una preocupación por la trazabilidad y la seguridad.

Beneficio principal

Mejora la trazabilidad y evita la exposición de los registros de otros usuarios.

Ampliación y validación de pruebas unitarias

La tercera mejora consistió en ampliar y validar las pruebas unitarias. En sistemas de autenticación, no basta con comprobar manualmente que el inicio de sesión funciona una sola vez. Es necesario verificar de forma repetible las reglas principales del sistema, especialmente aquellas relacionadas con validaciones, seguridad de contraseñas, tokens, recuperación de clave y auditoría.

Se revisaron y ejecutaron pruebas relacionadas con la validación de email, la validación de clave, la generación y validación de tokens, el flujo de login, la recuperación de contraseña, la actualización de clave, el uso de hash + salt y la auditoría. Estas pruebas permiten confirmar que las reglas críticas se mantienen en funcionamiento aunque el sistema cambie.

Una parte importante de esta mejora fue el uso de mocks. Las pruebas no deben depender de una base de datos real ni de un envío real de correos, porque eso las haría más lentas, frágiles y difíciles de repetir. Al simular repositorios, servicios de email y el consumo de tokens, las pruebas pueden ejecutarse de forma rápida y segura en un entorno académico o de desarrollo.

También se verificó que la auditoría personal filtrara únicamente para el usuario autenticado y que no se registraran datos sensibles, como contraseñas, hashes, salts o tokens. Esto convierte las pruebas en una herramienta no solo de calidad funcional, sino también de apoyo a las reglas de seguridad del sistema.

El principal beneficio de esta mejora es el aumento de la confianza técnica. Cuando existen pruebas automatizadas, el mantenimiento futuro se vuelve menos arriesgado porque cada cambio puede validarse rápidamente. Esto ayuda a detectar regresiones, documenta el comportamiento esperado y permite justificar académicamente que el sistema fue revisado más allá de una simple ejecución manual.

Beneficio principal

Aumenta la confianza técnica y reduce los riesgos durante el mantenimiento.

En conjunto, estas mejoras fortalecen tres dimensiones importantes del proyecto: la experiencia de usuario, la seguridad y la trazabilidad, y la mantenibilidad técnica. Esto permite presentar la práctica como una evolución controlada de un sistema funcional hacia una solución más clara, segura y verificable.

6. Reflexión sobre construcción y mantenimiento

La práctica evidencia una construcción incremental, pues el sistema puede entenderse como una evolución. En una primera etapa, se cuenta con un login básico que permite validar las credenciales en una base de datos. Posteriormente, se agregan validaciones de los campos obligatorios, del formato de email y de las restricciones de caracteres permitidos en la clave.

Después, el sistema incorpora protección de credenciales mediante hash + salt, recuperación de claves, doble autenticación mediante token, auditoría de accesos y pruebas unitarias. Esta evolución muestra cómo un sistema de software no suele construirse de forma completa desde el inicio, sino mediante incrementos que agregan valor y complejidad de manera gradual.

Cada una de esas etapas añade funcionalidad, pero también conlleva nuevas responsabilidades de mantenimiento. Por ejemplo, agregar 2FA mejora la seguridad, pero exige administrar tokens, su expiración, su envío por correo y la validación posterior al inicio de sesión. Incorporar la recuperación de clave facilita el uso del sistema, pero requiere controlar los tokens de recuperación, realizar la actualización segura de la contraseña y enviar mensajes adecuados al usuario. La auditoría aporta trazabilidad, pero también obliga a filtrar correctamente los registros para no exponer la información de otros usuarios. En otras palabras, cada mejora funcional conlleva nuevas reglas que deben comprenderse, probarse y mantener.

La construcción basada en componentes es fundamental para controlar esa complejidad. Separar responsabilidades, como la interfaz gráfica, las validaciones, la lógica de autenticación, el acceso a datos, el envío de email, la generación de tokens y la auditoría, permite que cada parte tenga un propósito claro. Cuando el código está organizado por responsabilidades, resulta más sencillo localizar un error, modificar una regla o agregar una nueva funcionalidad sin afectar el sistema en su totalidad. Esta separación también ayuda a que el proyecto resulte más comprensible para otros desarrolladores o estudiantes que deban revisarlo posteriormente.

En el caso de una aplicación con Tkinter, esta separación es especialmente importante. La interfaz gráfica debería encargarse de capturar datos, mostrar mensajes y permitir la navegación entre pantallas. La lógica de negocio debería decidir si un email es válido, si una contraseña coincide, si un token es correcto o si una auditoría debe registrarse. El acceso a los datos debería concentrarse en repositorios o en funciones que interactúen con SQL Server. Si estas responsabilidades se entrelazan, la interfaz se vuelve difícil de mantener y cualquier cambio visual puede terminar afectando las reglas críticas del sistema.

Las pruebas unitarias son necesarias para mantener la confianza cuando el sistema cambia. En un sistema de autenticación, pequeños errores pueden tener consecuencias importantes: aceptar un email inválido, permitir una clave con caracteres no definidos, validar incorrectamente un token o actualizar una contraseña sin protegerla adecuadamente. Al automatizar pruebas de validación, autenticación, tokens, recuperación de clave y auditoría, se reduce la dependencia de las pruebas manuales y se obtiene una forma de confirmar de manera repetible que las reglas principales siguen funcionando.

Además, las pruebas ayudan a documentar el comportamiento esperado del sistema. Un test que valida que un usuario inexistente no puede iniciar sesión ilustra una regla del sistema. Una prueba que confirma que la auditoría personal filtra por `id\_usuario` demuestra una decisión de seguridad. Una prueba que verifica que una contraseña no se almacena en texto plano deja

claro que la protección de las credenciales es una parte esencial del diseño. Por eso, las pruebas no solo sirven para detectar errores, sino también para especificar cómo debe comportarse el programa.

La seguridad debe considerarse desde la construcción, no como una etapa posterior. En esta práctica, decisiones como usar hash + salt, ocultar la contraseña en pantalla, evitar mostrar errores técnicos, controlar los tokens de 2FA, filtrar la auditoría por usuario autenticado y no exponer datos sensibles forman parte de la estructura del sistema. Si estas medidas se agregaran al final, probablemente serían más difíciles de integrar y podrían resultar inconsistentes. Construir con seguridad desde el inicio permite que las reglas de protección formen parte natural del diseño.

También es importante reconocer que la seguridad no depende de una sola función. No basta con guardar contraseñas protegidas si luego se le muestran al usuario final errores técnicos. No basta con generar tokens si no se controla su uso ni su expiración. No basta con registrar la auditoría si se permite consultar los registros de otros usuarios. La seguridad del sistema se basa en la combinación de varias decisiones pequeñas y consistentes a lo largo de su desarrollo.

Si el sistema crece sin estructura, resulta más difícil modificarlo, probarlo y corregir defectos. La falta de separación entre UI, lógica y datos aumenta el acoplamiento: una pantalla puede depender demasiado de una consulta SQL, una regla de validación puede duplicarse en varios lugares o una corrección puede romper una funcionalidad no relacionada. Este tipo de crecimiento desordenado hace que el mantenimiento sea lento y riesgoso, porque el equipo pierde claridad respecto de las consecuencias de cada cambio.

Por el contrario, un sistema construido con capas claras, funciones probables y responsabilidades bien definidas es más fácil de mantener. Permite mejorar la interfaz sin reescribir la autenticación, cambiar una consulta sin alterar la pantalla, agregar pruebas sin conectarse a servicios reales y revisar problemas de seguridad con mayor precisión. Esta práctica muestra que construir software no consiste únicamente en lograr que una aplicación funcione, sino en dejarla preparada para evolucionar de forma controlada, segura y comprensible.

7. Preguntas de reflexión

¿Qué parte del programa sería más difícil de mantener si el sistema creciera?

La parte más difícil de mantener sería la interacción entre la interfaz gráfica, la lógica de negocio y los servicios externos si esas responsabilidades empiezan a mezclarse en los

misimos archivos o funciones. En una aplicación con Tkinter, es común que los eventos de botones, los cambios de pantalla, las validaciones, las llamadas a la base de datos y los mensajes al usuario se concentren en una sola clase. Al principio, esto puede parecer práctico, pero cuando el sistema crece, resulta difícil identificar dónde se debe hacer un cambio y qué otras partes podrían verse afectadas.

En este caso, el flujo de autenticación incluye varias etapas: inicio de sesión, validación de campos, generación del token de 2FA, envío por email, verificación del token, recuperación de clave, cambio de contraseña y auditoría. Si todas esas reglas estuvieran mezcladas directamente con los componentes visuales, cualquier mejora en la interfaz podría vulnerar las reglas de seguridad o de acceso a datos. Por ejemplo, cambiar una pantalla podría afectar accidentalmente la forma en que se valida un token o se registra un evento de auditoría.

También sería difícil mantener el acceso a los datos si las consultas SQL estuvieran distribuidas entre distintos eventos de la interfaz. Esto haría más complicado cambiar la base de datos, ajustar una consulta, agregar filtros de seguridad o probar el sistema sin conectarse a SQL Server. Por eso es importante mantener los repositorios o servicios separados para usuarios, tokens y auditoría.

En resumen, la parte más delicada de mantener sería el flujo completo de autenticación si no se mantiene una separación clara entre la interfaz, las reglas de negocio, los servicios externos y la base de datos. A medida que el sistema crece, la mantenibilidad depende menos de que el código funcione hoy y más de que sea fácil de entender, modificar y probar mañana.

¿Qué riesgos existen al guardar la clave en texto plano?

Guardar la clave en texto plano representa uno de los riesgos más graves en un sistema de autenticación. Si una persona no autorizada obtiene acceso a la base de datos, podría leer directamente las contraseñas de los usuarios. Esto no solo compromete el sistema actual, sino también otros servicios en los que los usuarios hayan reutilizado la misma contraseña.

El impacto de una filtración sería mucho mayor, ya que no habría una barrera técnica que proteja las credenciales. En cambio, cuando se usa hash + salt, la aplicación no almacena la contraseña original, sino un resultado criptográfico que permite verificarla sin conocerla directamente. El salt agrega una capa adicional porque impide que dos usuarios con la misma contraseña tengan el mismo hash.

Además, guardar contraseñas en texto plano incumple las buenas prácticas de seguridad y puede generar problemas éticos, legales o reputacionales. Un sistema que maneja credenciales debe asumir que la base de datos podría ser atacada o filtrada en algún momento, por lo que debe reducir el daño potencial desde el diseño.

Otro riesgo importante es que el personal técnico con acceso legítimo a la base de datos también podría ver las contraseñas si estuvieran en texto plano. Esto rompe el principio de

mínimo conocimiento: ni administradores ni desarrolladores deberían necesitar conocer la clave real de un usuario para operar o mantener el sistema.

Por estas razones, una contraseña nunca debería guardarse ni mostrarse en texto plano. El uso de hash + salt, junto con validaciones, auditoría y un manejo seguro de errores, ayuda a construir un sistema más responsable y seguro.

¿Por qué conviene separar la interfaz gráfica del acceso a datos?

Separar la interfaz gráfica del acceso a datos conviene porque reduce el acoplamiento entre dos partes del sistema que cambian por razones distintas. La interfaz cambia cuando se mejora la experiencia del usuario, se reorganizan las pantallas o se actualiza el diseño visual. El acceso a los datos cambia cuando se modifica el esquema de la base de datos, se optimizan las consultas o se ajustan las reglas de persistencia. Si ambas responsabilidades están mezcladas, cualquier cambio pequeño puede resultar riesgoso.

En una aplicación Tkinter, la interfaz debería encargarse principalmente de capturar las entradas del usuario, mostrar mensajes y permitir la navegación entre pantallas. La lógica de acceso a SQL Server debería estar encapsulada en repositorios o servicios especializados. De esta manera, la pantalla de login no necesita saber cómo consultar una tabla, abrir una conexión ni parametrizar una consulta.

Esta separación también facilita las pruebas unitarias. Si la lógica depende directamente de ventanas, botones o campos visuales, probarla requiere abrir la aplicación o simular eventos gráficos. En cambio, si la lógica está en servicios independientes, se pueden usar mocks para simular la base de datos y validar casos como usuario inexistente, contraseña incorrecta, token inválido o recuperación de clave sin depender de SQL Server real.

Otro beneficio es la posibilidad de cambiar una parte del sistema con menor impacto. Si en el futuro se reemplazara Tkinter por otra interfaz, como una aplicación web, la lógica de autenticación y los repositorios podrían reutilizarse. De igual forma, si se cambiara SQL Server por otro motor de base de datos, la interfaz no tendría por qué modificarse si el acceso a los datos estuviera bien encapsulado.

Por lo tanto, separar la interfaz del acceso a datos mejora la claridad, la mantenibilidad, la seguridad y la capacidad de prueba del sistema.

¿Qué parte debería automatizarse mediante pruebas unitarias?

Deberían automatizarse principalmente las partes críticas, repetibles y aislables del sistema. En primer lugar, las validaciones de entrada son candidatas naturales para pruebas unitarias: email vacío, email inválido, clave vacía, clave con caracteres no permitidos y token con formato incorrecto. Estas reglas son funciones relativamente pequeñas, pero son importantes porque evitan que datos inválidos lleguen a capas más sensibles.

También debe automatizarse la lógica de seguridad de las contraseñas. Las pruebas deben confirmar que la clave no se guarda en texto plano, que el hash cambia al modificar el salt, que una contraseña correcta puede verificarse mediante su hash y que una contraseña incorrecta falla. Este tipo de pruebas ayuda a evitar regresiones en una parte crítica del sistema.

El flujo de login también debe probarse con mocks. Es importante validar casos como usuario inexistente, usuario inactivo, clave incorrecta y login correcto. Además, se debe comprobar que un inicio de sesión correcto no abre directamente el sistema, sino que avanza al flujo de 2FA. Esto confirma que la autenticación en dos pasos no se omite por error.

La generación y validación de tokens también deben cubrirse. Las pruebas deben verificar que el token tenga la longitud esperada, el formato correcto, que un token válido sea aceptado y que uno inválido sea rechazado. Si existe expiración o estado de uso, también deberían probarse tokens expirados o ya utilizados.

La recuperación de la clave es otro aspecto importante. Deben probarse email inexistente, email válido, generación de token de recuperación, envío de correo simulado, token incorrecto, token correcto y actualización de la contraseña con hash + salt. Ninguna de estas pruebas debe enviar correos reales ni conectarse a una base de datos real.

Finalmente, la auditoría debe automatizarse para comprobar que los eventos se registran correctamente, que no se registren contraseñas, hashes, salts ni tokens, y que la auditoría personal filtre únicamente por el usuario autenticado.

¿Qué cambiaría si este sistema fuera para producción real?

Si este sistema fuera para producción real, el primer cambio sería fortalecer la gestión de la configuración y de los secretos. Las credenciales de base de datos, SMTP y cualquier dato sensible deberían gestionarse mediante variables de entorno, servicios de secretos o herramientas especializadas. Nunca deberían quedar quemadas en el código ni mostrarse en los mensajes de error.

También habría que reforzar la autenticación. Aunque el sistema ya usa hash + salt y 2FA, en producción sería recomendable aplicar políticas como bloqueo temporal por intentos fallidos, límites de frecuencia, expiración estricta de tokens, invalidación de tokens usados y reglas más robustas para contraseñas. Además, el token no debería mostrarse en la consola, salvo en un modo de desarrollo claramente controlado.

La auditoría también debería protegerse mejor. En un entorno real, los registros de auditoría deberían estar disponibles solo para el titular de la información o para los roles administrativos autorizados. Además, los eventos deberían ser lo suficientemente detallados para la trazabilidad, pero sin incluir datos sensibles.

Otro cambio importante sería el manejo de errores. El usuario final debe recibir mensajes claros y generales, mientras que los detalles técnicos deben registrarse en logs seguros. No se deben mostrar excepciones, cadenas de conexión, consultas SQL, credenciales de SMTP ni información interna del sistema.

También sería necesario agregar monitoreo, logging técnico y alertas. Un sistema de autenticación en producción debe permitir detectar patrones anómalos, intentos de acceso repetidos, fallas de correo electrónico o problemas de conexión con la base de datos.

En cuanto a la calidad, el sistema debería ejecutarse con integración continua, pruebas automatizadas y revisión de seguridad antes de cada despliegue. Además de las pruebas unitarias, convendría agregar pruebas de integración contra una base de datos de prueba, pruebas de flujos completos y la revisión de dependencias.

Finalmente, debería revisarse el despliegue completo: permisos de la base de datos, cifrado de las comunicaciones, respaldos, manejo de logs, rotación de secretos y políticas de acceso. Llevar un sistema académico a producción implica no solo que funcione, sino también que sea seguro, observable, mantenible y resistente a fallos.